VISVESVARAYA TECHNOLOGICAL UNIVERSITY

"JnanaSangama", Belgaum -590014, Karnataka.

LAB REPORT

On

ANALYSIS AND DESIGN OF ALGORITHMS (23CS4PCADA)

Submitted by

KRISHNA LATH (1BF24CS149)

in partial fulfillment for the award of the degree of
BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING

B.M.S. COLLEGE OF ENGINEERING
(Autonomous Institution under VTU)
BENGALURU-560019
February to June 2026

CERTIFICATE

B. M. S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering

This is to certify that the Lab work entitled **"ANALYSIS AND DESIGN OF ALGORITHMS"** carried out by KRISHNA LATH**(1BF24CS149)**, who is bonafide student of **B. M. S. College of Engineering**. It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum during the year 2025-26. The Lab report has been approved as it satisfies the academic requirements in respect of Analysis and Design of Algorithms Lab - **(23CS4PCADA)** work prescribed for the said degree.

Dr. Manjula S
Assistant Professor
Department of CSE
BMSCE, Bengaluru

Dr. Kavitha Sooda
Professor and Head
Department of CSE
BMSCE, Bengaluru

Index Sheet

Course outcomes:

CO1	Analyze time complexity of algorithms using asymptotic notations.
CO2	Apply various algorithm design techniques for the given problem.
CO3	Evaluate the appropriate algorithm/design technique for solving a given problem.
CO4	Conduct practical experiments by applying appropriate algorithm design technique to solve problems.

Lab program 1:

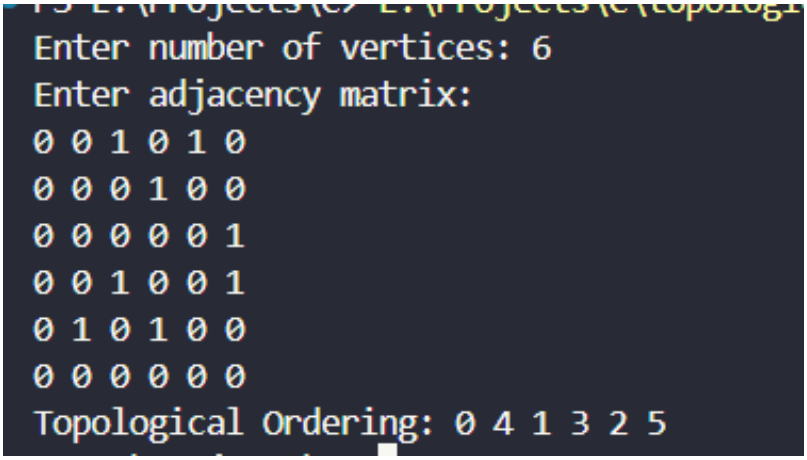
CODE:

```
#include <stdio.h>

#define MAX 100

int graph[MAX][MAX];
int indegree[MAX];
int queue[MAX];
int front = 0, rear = 0;

void topologicalSort(int n)
{
   int i, j, count = 0;

   // Calculate indegree of each vertex
   for(i = 0; i < n; i++)
   {
      indegree[i] = 0;
   }

   for(i = 0; i < n; i++)
   {
      for(j = 0; j < n; j++)
      {
         if(graph[i][j] == 1)
         {
            indegree[j]++;
         }
      }
   }

   for(i = 0; i < n; i++)
   {
      if(indegree[i] == 0)
      {
         queue[rear++] = i;
      }
   }

   printf("Topological Ordering: ");

   while(front < rear)
   {
      int vertex = queue[front++];
      printf("%d ", vertex);
      count++;

      // Reduce indegree of adjacent vertices
      for(i = 0; i < n; i++)
      {
         if(graph[vertex][i] == 1)
         {
```

```
            indegree[i]--;

            if(indegree[i] == 0)
            {
               queue[rear++] = i;
            }
         }
      }
   }

   if(count != n)
   {
      printf("\nGraph contains a cycle. Topological ordering not possible.");
   }
}

int main()
{
   int n, i, j;

   printf("Enter number of vertices: ");
   scanf("%d", &n);

   printf("Enter adjacency matrix:\n");

   for(i = 0; i < n; i++)
   {
      for(j = 0; j < n; j++)
      {
         scanf("%d", &graph[i][j]);
      }
   }

   topologicalSort(n);
   return 0;
}
```

OUTPUT:

```
Enter number of vertices: 4
Enter adjacency matrix:
0 1 1 0
0 0 1 1
0 0 0 1
0 0 0 0
Topological Ordering: 0 1 2 3
```



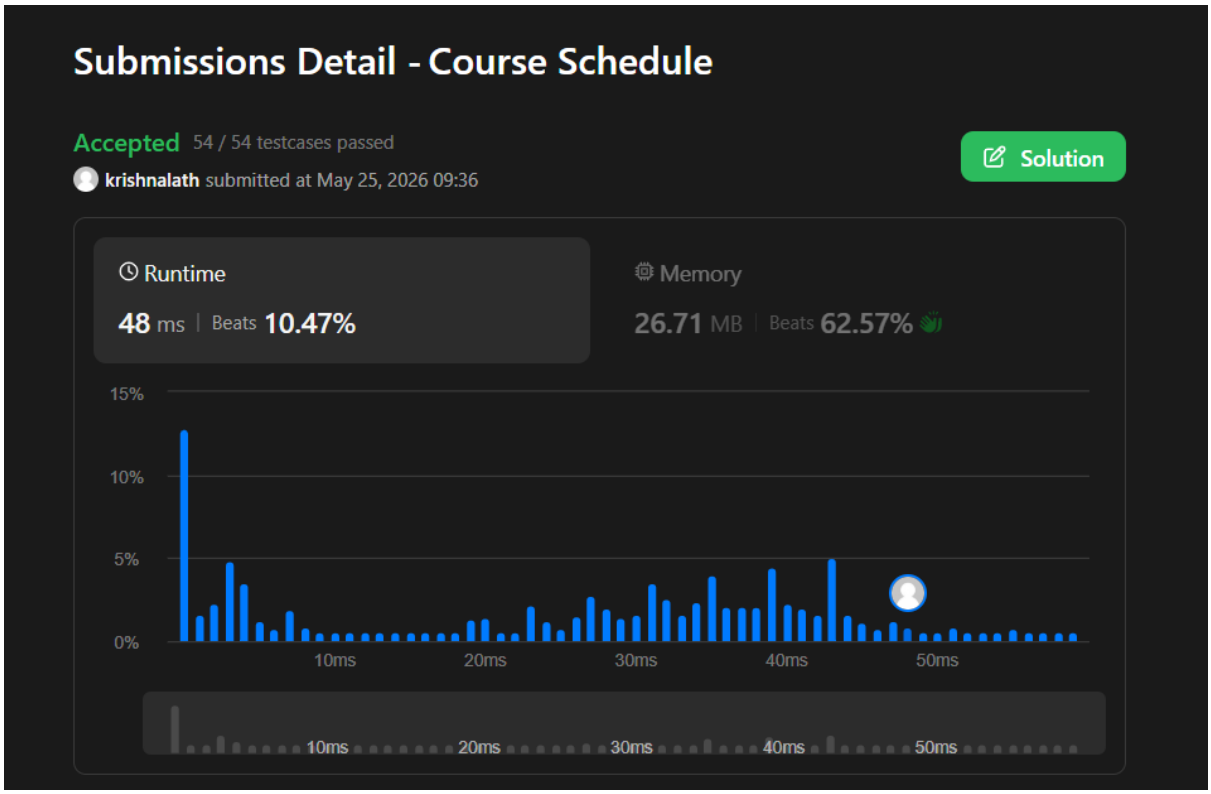
LEETCODE 207.Course Schedule

CODE:

```
#include <stdbool.h>
#include <stdlib.h>

bool canFinish(int numCourses, int** prerequisites, int prerequisitesSize, int* prerequisitesColSize) {

    int graph[2000][2000] = {0};
    int indegree[2000] = {0};


    for(int i = 0; i < prerequisitesSize; i++) {
        int a = prerequisites[i][0];
        int b = prerequisites[i][1];

        graph[b][a] = 1;   // b -> a
        indegree[a]++;
    }


    int queue[2000];
    int front = 0, rear = 0;


    for(int i = 0; i < numCourses; i++) {
        if(indegree[i] == 0) {
            queue[rear++] = i;
        }
    }

    int count = 0;


    while(front < rear) {
        int node = queue[front++];
        count++;
```

```
        for(int i = 0; i < numCourses; i++) {
            if(graph[node][i] == 1) {
                indegree[i]--;

                if(indegree[i] == 0) {
                    queue[rear++] = i;
                }
            }
        }
    }

    return count == numCourses;
}
```

OUTPUT:

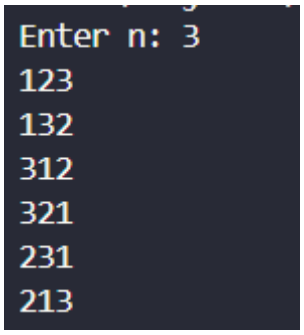


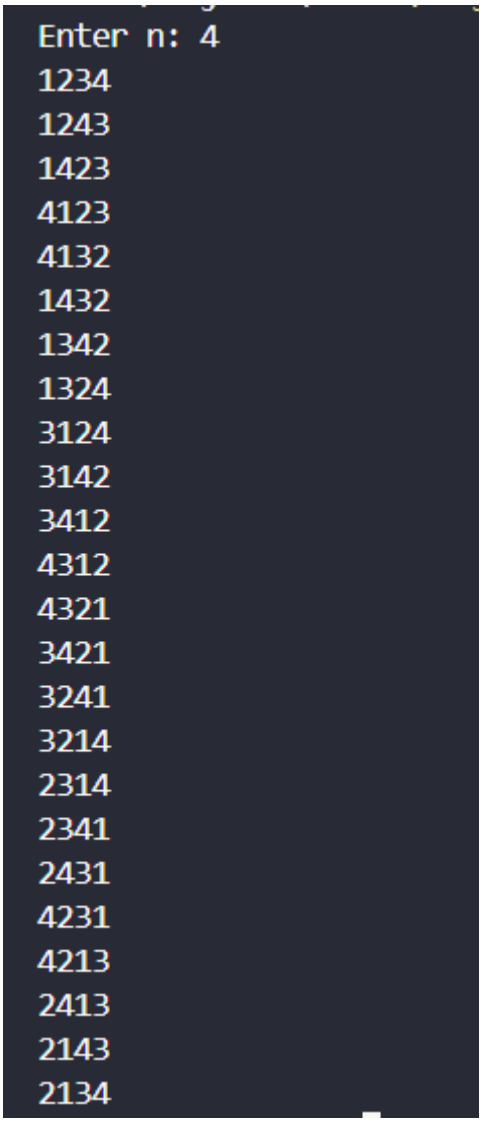
Lab program 2:

CODE:

```
#include <stdio.h>

int mobile(int p[],int d[],int n){
   int m=-1;
   for(int i=0;i<n;i++){
      if(d[i]==-1 && i>0 && p[i]>p[i-1] && p[i]>m)
         m=p[i];
      if(d[i]==1 && i<n-1 && p[i]>p[i+1] && p[i]>m)
         m=p[i];
   }
   return m;
}

int main(){
   int n;
   printf("Enter n: ");
   scanf("%d",&n);
   int p[n],d[n];
   for(int i=0;i<n;i++){
      d[i]=-1;
      p[i]=i+1;
   }
   while(1){
      for(int i=0;i<n;i++)
         printf("%d",p[i]);
      printf("\n");
      int m=mobile(p,d,n);
      if(m==-1) break;
      int pos;
      for(int i=0;i<n;i++){
         if(p[i]==m)
            pos = i;
      }
      int next = pos+d[pos];
      int temp = p[pos];
      p[pos]=p[next];
      p[next]=temp;
      temp=d[pos];
      d[pos]=d[next];
      d[next]=temp;

      for(int i=0;i<n;i++){
         if(p[i]>m)
            d[i]=-d[i];
      }
   }
   return 0;
}
```

OUTPUT:


Enter n: 3
123
132
312
321
231
213


Enter n: 4
1234
1243
1423
4123
4132
1432
1342
1324
3124
3142
3412
4312
4321
3421
3241
3214
2314
2341
2431
4231
4213
2413
2143
2134

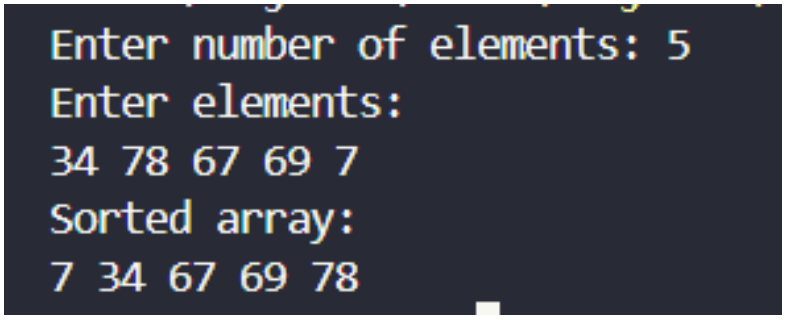
Lab program 3:

CODE:

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

void merge(int arr[], int low, int mid, int high) {
   int i = low, j = mid + 1, k = low;
   int temp[high+1];

   while (i <= mid && j <= high) {
      if (arr[i] < arr[j])
         temp[k++] = arr[i++];
      else
         temp[k++] = arr[j++];
   }

   while (i <= mid)
      temp[k++] = arr[i++];

   while (j <= high)
      temp[k++] = arr[j++];

   for (i = low; i <= high; i++)
      arr[i] = temp[i];
}

void mergeSort(int arr[], int low, int high) {
   if (low < high) {
      int mid = (low + high) / 2;

      mergeSort(arr, low, mid);
      mergeSort(arr, mid + 1, high);
      merge(arr, low, mid, high);
   }
}

int main() {
   int n = 60000;
   int arr[n];

   srand(time(NULL));

   for (int i = 0; i < n; i++) {
      arr[i] = rand() % 2*n;
   }

   clock_t start, end;
   double cpu_time;

   start = clock();

   mergeSort(arr, 0, n - 1);
```

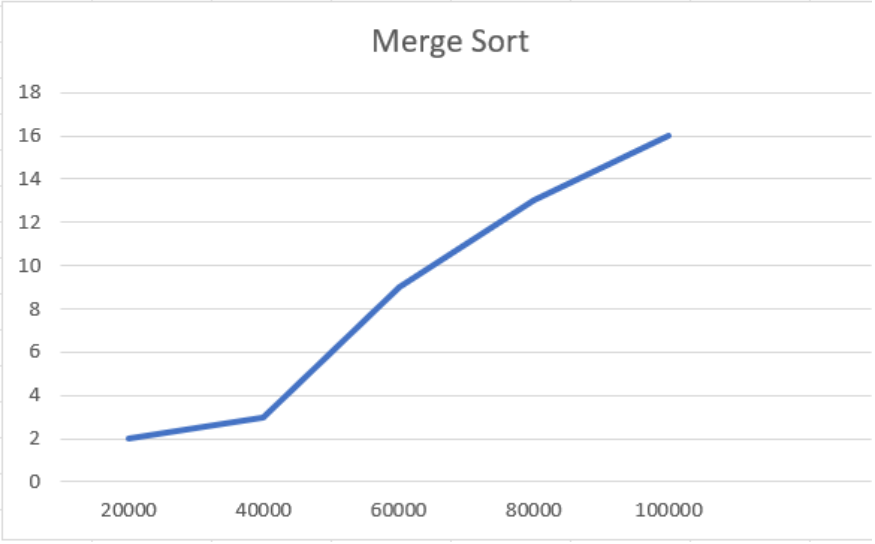
```
    end = clock();
    cpu_time = (((double)(end - start)) / CLOCKS_PER_SEC)*1000.0;
    printf("Time taken = %f miliseconds\n", cpu_time);
    return 0;
}
```

OUTPUT:



GRAPH:

Input Size(n)	Time (ms)
20000	2
40000	3
60000	9
80000	13
100000	16



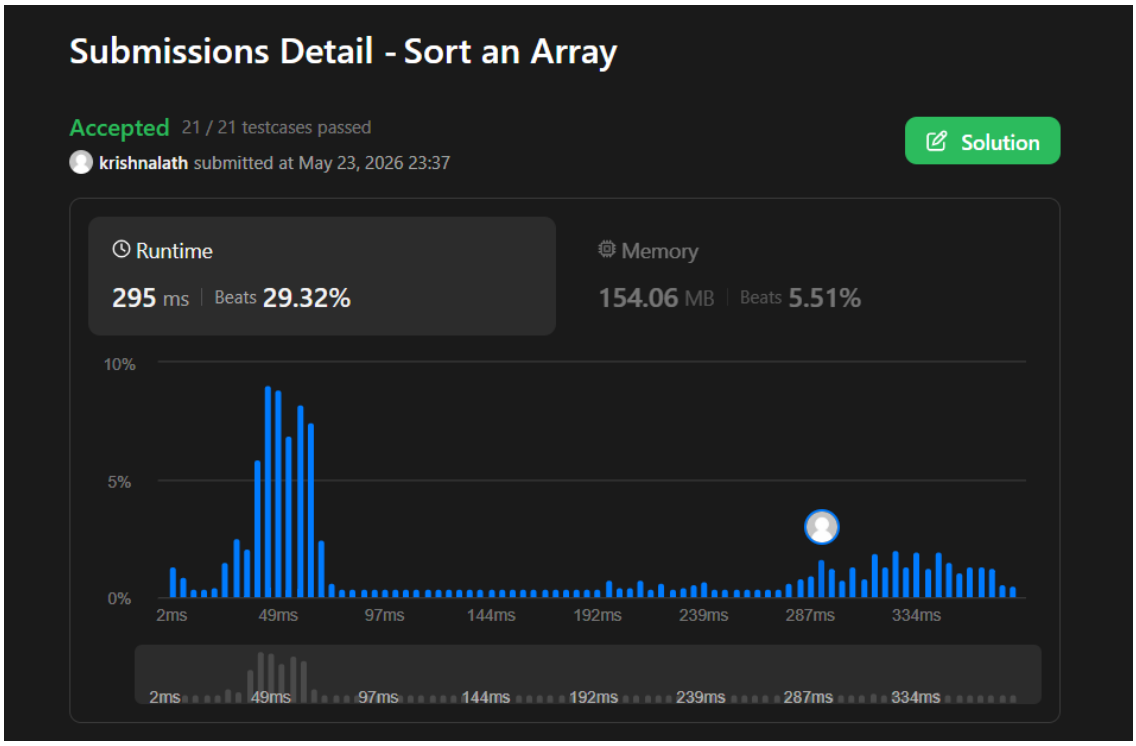
LEETCODE 912: Sort an Array

CODE:

```
#include <stdlib.h>

void merge(int* nums, int left, int mid, int right) {
    int i, j, k;
    int n1 = mid - left + 1;
    int n2 = right - mid;

    int* L = (int*)malloc(n1 * sizeof(int));
    int* R = (int*)malloc(n2 * sizeof(int));

    for (i = 0; i < n1; i++) {
        L[i] = nums[left + i];
    }
    for (j = 0; j < n2; j++) {
        R[j] = nums[mid + 1 + j];
    }

    i = 0;
    j = 0;
    k = left;

    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) {
            nums[k] = L[i];
            i++;
        } else {
            nums[k] = R[j];
            j++;
        }
        k++;
    }

    while (i < n1) {
        nums[k] = L[i];
        i++;
        k++;
    }

    while (j < n2) {
        nums[k] = R[j];
        j++;
        k++;
    }

    free(L);
    free(R);
}

void mergeSort(int* nums, int left, int right) {
    if (left < right) {
```

```
        int mid = left + (right - left) / 2;

        mergeSort(nums, left, mid);
        mergeSort(nums, mid + 1, right);

        merge(nums, left, mid, right);
    }
}

int* sortArray(int* nums, int numsSize, int* returnSize) {
    mergeSort(nums, 0, numsSize - 1);
    *returnSize = numsSize;
    return nums;
}
```

OUTPUT:



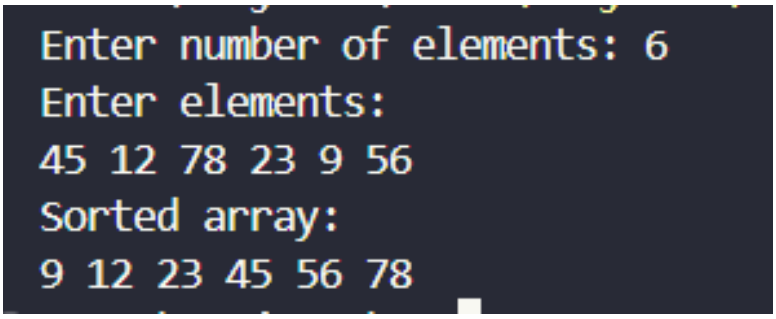
Lab program 4:

CODE:

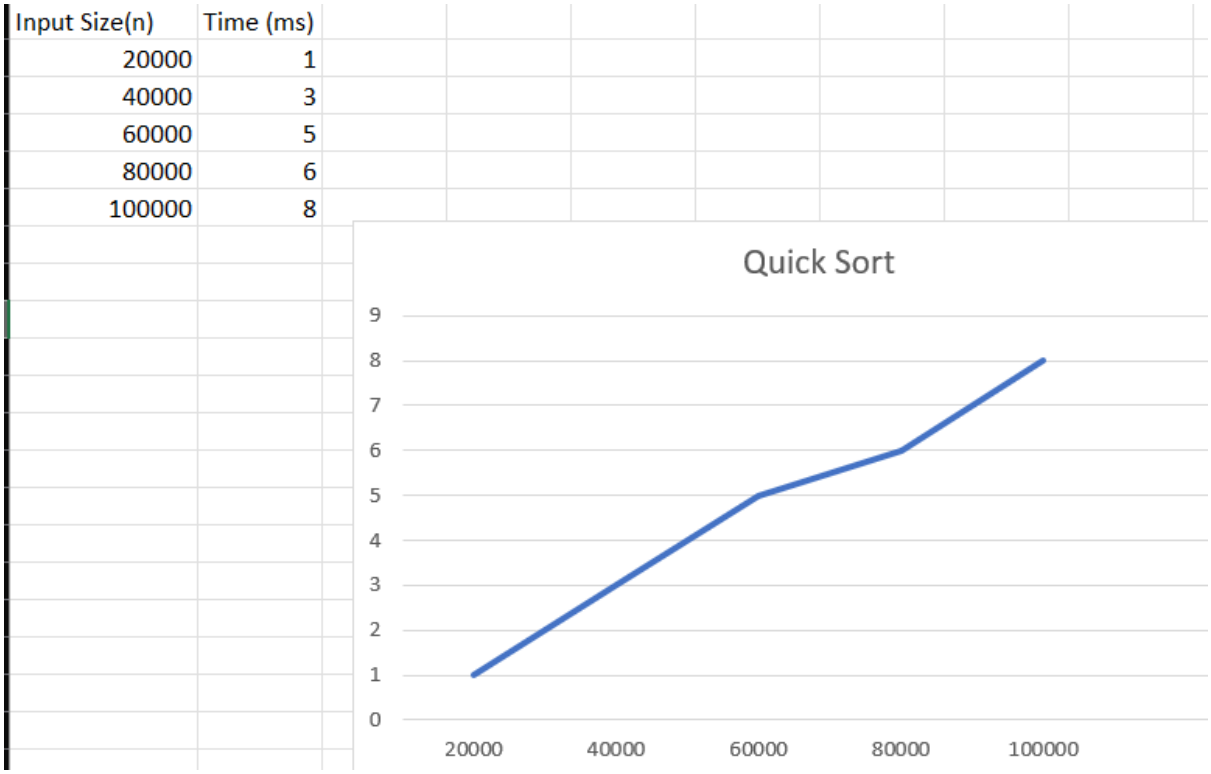
```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
int partition(int low,int high,int arr[]){
   int i=low;
   int j=high+1;
   int pivot=arr[low];
   while(i<j){
      do{
         i++;
      }while(i<=high && arr[i]<=pivot);
      do{
         j--;
      }while(arr[j]>pivot);
      if(i<j){
         int temp=arr[i];
         arr[i]=arr[j];
         arr[j]=temp;
      }
   }
  int temp2 = arr[j];
  arr[j]=pivot;
  arr[low]=temp2;
  return j;
}

void quickSort(int low,int high,int arr[]){
   if(low<high){
      int p=partition(low,high,arr);
      quickSort(low,p-1,arr);
      quickSort(p+1,high,arr);
   }
}

int main() {
   int n = 75000;
   int arr[n];

   srand(time(NULL));

   for (int i = 0; i < n; i++) {
      arr[i] = rand() % (2 * n);
   }

   clock_t start, end;
   double cpu_time;

   start = clock();

   quickSort(0, n - 1, arr);
```

```
    end = clock();
    cpu_time = ((double)(end - start) / CLOCKS_PER_SEC) * 1000.0;
    printf("Time taken = %.3f milliseconds\n", cpu_time);
    return 0;
}
```

OUTPUT:



GRAPH:

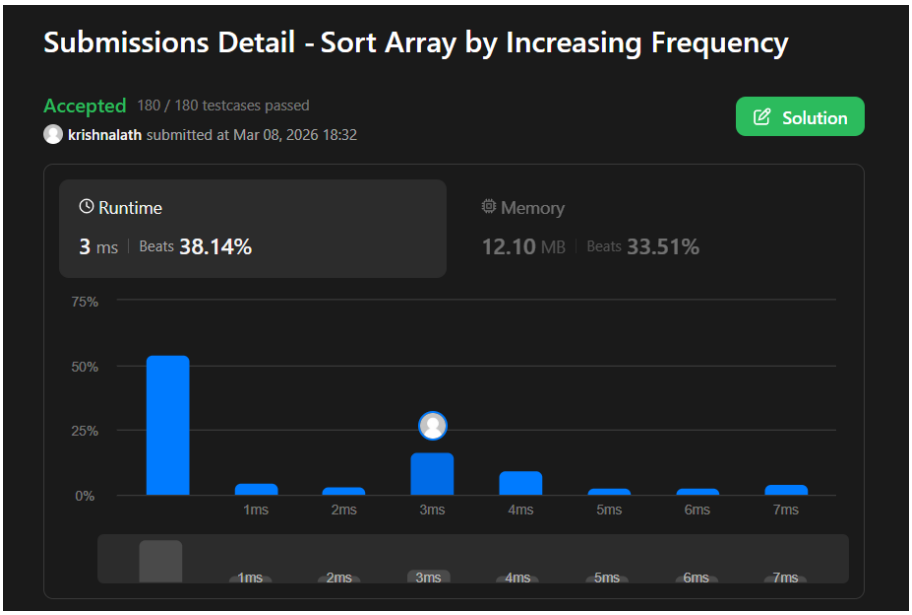


LEETCODE 1636: Sort Array By Increasing Frequency

CODE:

```
int* frequencySort(int* nums, int numsSize, int* returnSize) {

    int freq[201] = {0};

    for(int i = 0; i < numsSize; i++){
        freq[nums[i] + 100]++;
    }

    for(int i = 0; i < numsSize-1; i++){
        for(int j = i+1; j < numsSize; j++){

            if(freq[nums[i]+100] > freq[nums[j]+100] ||
              (freq[nums[i]+100] == freq[nums[j]+100] && nums[i] < nums[j])){

                int temp = nums[i];
                nums[i] = nums[j];
                nums[j] = temp;
            }
        }
    }

    *returnSize = numsSize;
    return nums;
}
```

OUTPUT:



Lab program 5:

CODE:

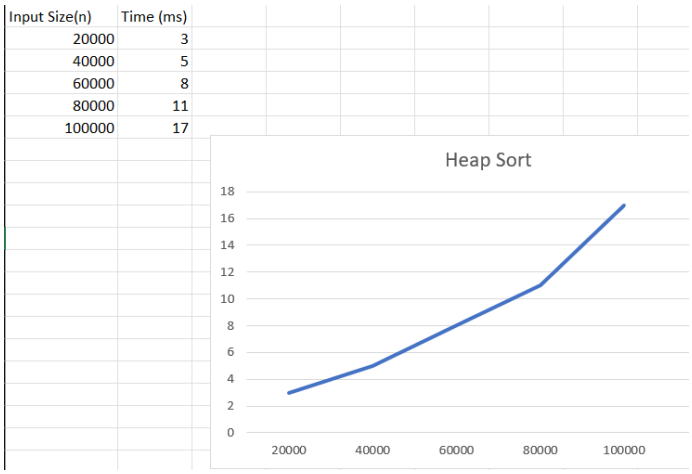
```
#include <stdio.h>
#include <time.h>
#include <stdbool.h>

void heapify(int H[], int n) {

   for (int i = n / 2; i >= 1; i--) {

      int v = H[i];
      int k = i;
      bool heap = false;

      while (!heap && 2 * k <= n) {

         int j = 2 * k;
         if (j < n && H[j] < H[j + 1])
            j = j + 1;

         if (v >= H[j])
            heap = true;
         else {
            H[k] = H[j];
            k = j;
         }
      }

      H[k] = v;
   }
}

void heapsort(int H[], int n) {

   heapify(H, n);

   for (int i = n; i >= 2; i--) {
      int temp = H[1];
      H[1] = H[i];
      H[i] = temp;
      heapify(H, i - 1);
   }
}

int main() {

   int n;

   printf("Enter no. of elements: ");
   scanf("%d", &n);

   int H[n + 1];
```

```
    printf("Enter the elements:\n");

    for (int i = 1; i <= n; i++)
        scanf("%d", &H[i]);

    clock_t start, end;

    start = clock();

    heapsort(H, n);

    end = clock();

    double cpu_time_used =
        (((double)(end - start)) / CLOCKS_PER_SEC) * 1000;

    printf("Sorted Elements:\n");

    for (int i = 1; i <= n; i++)
        printf("%d ", H[i]);

    printf("\n");

    printf("Input size: %d\n", n);
    printf("Execution time: %.5f ms\n", cpu_time_used);
}
```

OUTPUT:

```
Enter no. of elements: 7
Enter the elements:
23 43 10 67 100 15 50
Sorted Elements:
10 15 23 43 50 67 100
Input size: 7
Execution time: 0.00000 ms
```

GRAPH:

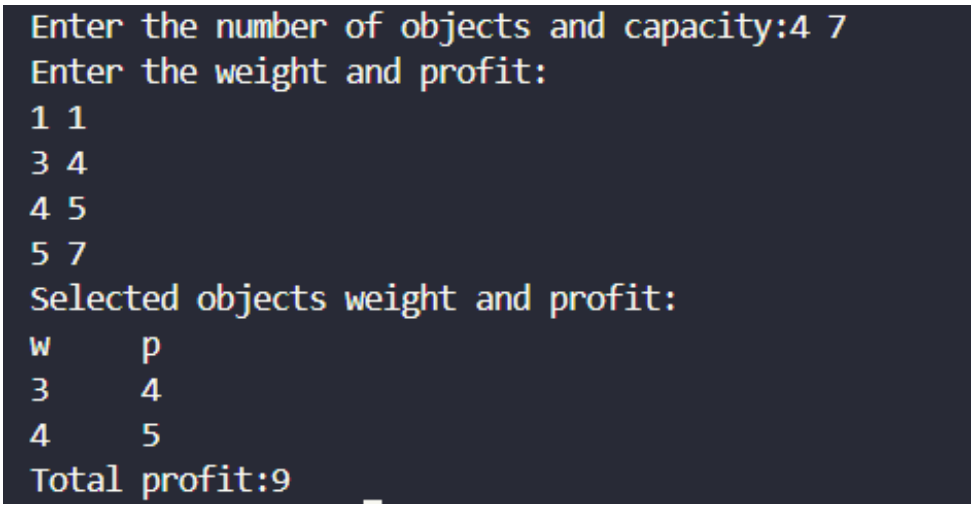


Lab program 6:

CODE:

```
#include<stdio.h>
int max(int a,int b){
   return (a>b)?a:b;
}
void main(){
   int n,m;
   printf("Enter the number of objects and capacity:");
   scanf("%d %d",&n,&m);
   int v[n+1][m+1];
   int p[n+1],w[n+1],x[n+1];
   printf("Enter the weight and profit:\n");
   for(int i=1;i<=n;i++){
      scanf("%d %d",&w[i],&p[i]);
   }
   for(int i=0;i<=n;i++){
      x[i]=0;
   }
   for(int i=0;i<=n;i++){
      for(int j=0;j<=m;j++){
         if(i==0 || j==0) v[i][j]=0;
         else if(w[i]<=j) v[i][j]=max(v[i-1][j],p[i]+v[i-1][j-w[i]]);
         else v[i][j]=v[i-1][j];
      }
   }
   int i=n;
   int j=m;
   while(i>0 && j>0){
      if(v[i][j]!=v[i-1][j]){
         x[i]=1;
         j=j-w[i];
      }
      i=i-1;
   }
   printf("Selected objects weight and profit:\n");
   printf("%-5s %-5s","w","p");
   for(int i=0;i<=n;i++){
      if(x[i]==1) printf("\n%-5d %-5d",w[i],p[i]);
   }
   printf("\nTotal profit:%d",v[n][m]);
}
```

OUTPUT:

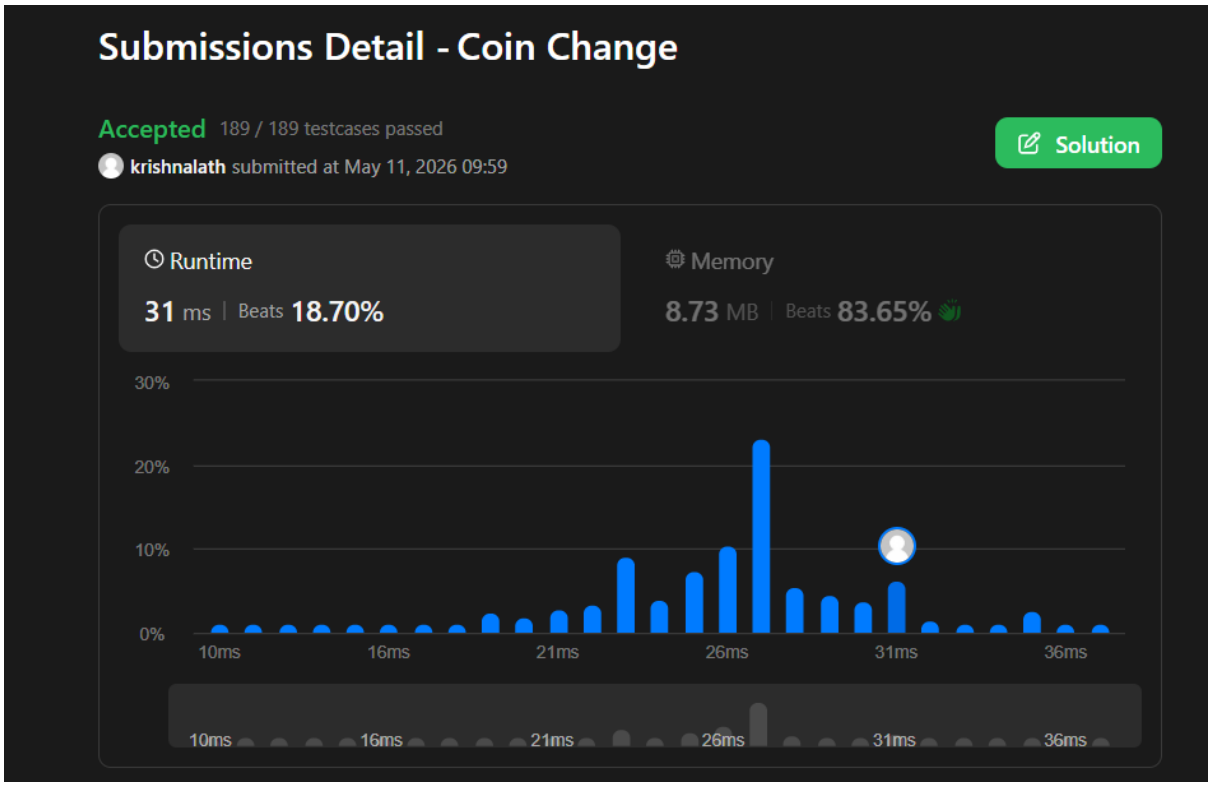


LEETCODE 322: Coin Change

CODE:

```
int coinChange(int* coins, int coinsSize, int amount) {
    int dp[amount + 1];
    for (int i = 0; i <= amount; i++) {
        dp[i] = amount + 1;
    }

    dp[0] = 0;
    for (int i = 1; i <= amount; i++) {
        for (int j = 0; j < coinsSize; j++) {
            if (coins[j] <= i) {
                if (dp[i - coins[j]] + 1 < dp[i]) {
                    dp[i] = dp[i - coins[j]] + 1;
                }
            }
        }
    }

    return (dp[amount] > amount) ? -1 : dp[amount];
}
```

OUTPUT:

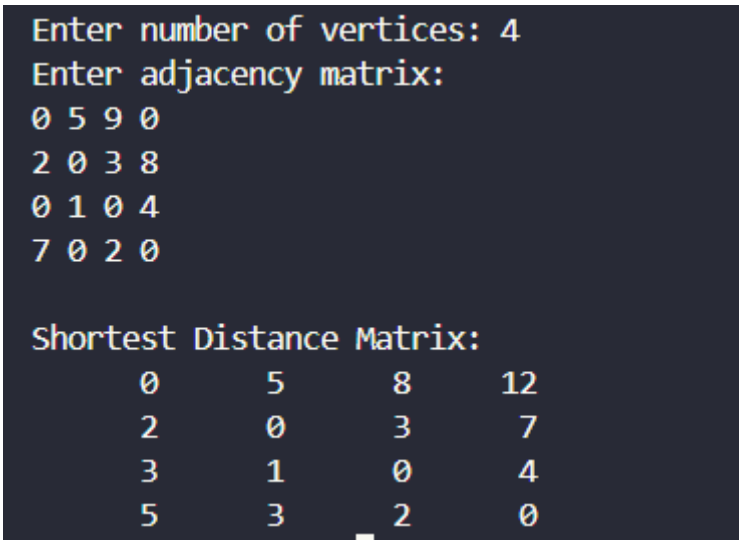

Submissions Detail - Coin Change
Accepted 189 / 189 testcases passed
krishnalath submitted at May 11, 2026 09:59
Solution
Runtime
31 ms | Beats 18.70%
Memory
8.73 MB | Beats 83.65%
30%
20%
10%
0%
10ms
16ms
21ms
26ms
31ms
36ms

Lab program 7:

CODE:

```
#include<stdio.h>
#include<limits.h>

int main() {
   int n;
   printf("Enter number of vertices : ");
   scanf("%d", &n);

   int a[n][n];
   printf("Enter adjacency matrix:\n");
   for(int i = 0; i < n; i++){
      for(int j = 0; j < n; j++){
         scanf("%d", &a[i][j]);
      }
   }
   for(int i=0;i<n;i++){
      for(int j=0;j<n;j++){
         if(i!=j && a[i][j]==0)
            a[i][j] = INT_MAX;
      }
   }
   for(int i=0;i<n;i++){
      for(int j=0;j<n;j++){
         for(int k=0;k<n;k++){
            if(a[i][k]!=INT_MAX && a[k][i] != INT_MAX){
               if(a[i][k]+a[k][j] < a[i][j])
                  a[i][j] = a[i][k] + a[k][j];
            }
         }
      }
   }
   printf("Shortest Path: \n");
   for(int i=0;i<n;i++){
      for(int j=0;j<n;j++){
         if(a[i][j] == INT_MAX)
            printf("INFINITY");
         else
            printf("%d",a[i][j]);
      }
   }
}
```

OUTPUT:



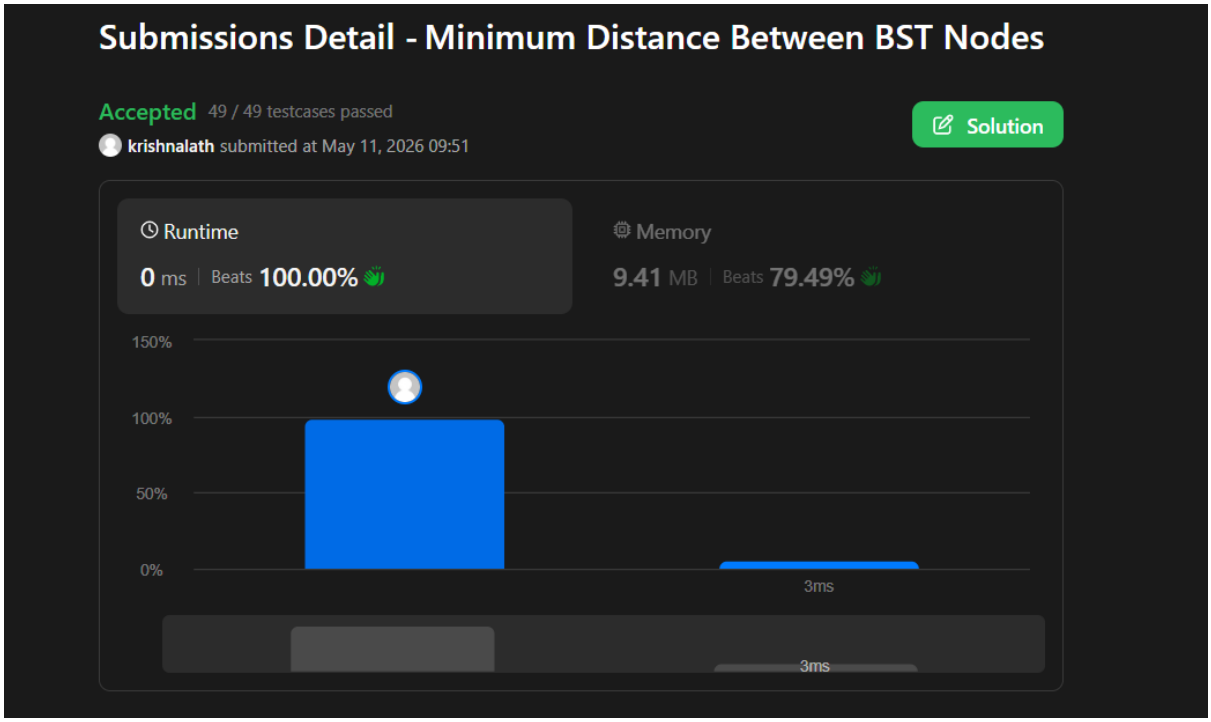
LEETCODE 783: Minimum Distance Between BST Nodes

CODE:

```
/**
 * Definition for a binary tree node.
 * struct TreeNode {
 *     int val;
 *     struct TreeNode *left;
 *     struct TreeNode *right;
 * };
 */

int prev;
bool hasPrev = false;
int ans;

void inorder(struct TreeNode* root) {
    if (root == NULL)
        return;

    inorder(root->left);

    if (hasPrev) {
        int diff = root->val - prev;
        if (diff < ans)
            ans = diff;
    }

    prev = root->val;
    hasPrev = true;
```

```
    inorder(root->right);
}

int minDiffInBST(struct TreeNode* root) {
    ans = INT_MAX;
    hasPrev = false;

    inorder(root);

    return ans;
}
```

OUTPUT:



Lab program 8:

8(a)
CODE:

```
#include <stdio.h>
#define MAX 10
#define INF 999

int parent[MAX];

int find(int x)
{
   while(parent[x] != x)
      x = parent[x];

   return x;
}

void unionSet(int a, int b)
{
   parent[find(a)] = find(b);
}

int main()
{
   int cost[MAX][MAX];
   int n, i, j;
   int min, a, b;
   int edges = 0, minCost = 0;

   printf("Enter number of vertices: ");
   scanf("%d", &n);

   printf("Enter adjacency matrix:\n");

   // Read matrix
   for(i = 0; i < n; i++)
   {
      for(j = 0; j < n; j++)
      {
         scanf("%d", &cost[i][j]);

         // 0 means no edge
         if(cost[i][j] == 0)
            cost[i][j] = INF;
      }
   }

   for(i = 0; i < n; i++)
   {
      parent[i] = i;
   }

   printf("Edges in MST:\n");
```

```
    while(edges < n - 1)
    {
        min = INF;

        for(i = 0; i < n; i++)
        {
            for(j = i + 1; j < n; j++)
            {
                if(cost[i][j] < min)
                {
                    min = cost[i][j];
                    a = i;
                    b = j;
                }
            }
        }


        if(find(a) != find(b))
        {
            printf("%d - %d = %d\n", a, b, min);

            minCost += min;

            unionSet(a, b);

            edges++;
        }

        cost[a][b] = cost[b][a] = INF;
    }

    printf("Minimum Cost = %d\n", minCost);

    return 0;
}
```

OUTPUT:

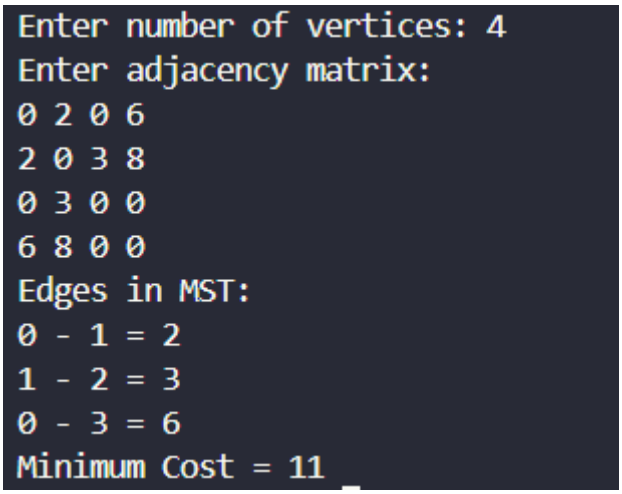
```
Enter number of vertices: 4
Enter adjacency matrix:
0 2 0 6
2 0 3 8
0 3 0 0
6 8 0 0
Edges in MST:
0 - 1 = 2
1 - 2 = 3
0 - 3 = 6
Minimum Cost = 11
```

8(b)
CODE:

```
#include <stdio.h>

#define MAX 10
#define INF 999

int main()
{
   int cost[MAX][MAX];
   int visited[MAX] = {0};

   int n, i, j;
   int min, a, b;
   int edges = 0;
   int minCost = 0;

   printf("Enter number of vertices: ");
   scanf("%d", &n);

   printf("Enter adjacency matrix:\n");

   for(i = 0; i < n; i++)
   {
      for(j = 0; j < n; j++)
      {
         scanf("%d", &cost[i][j]);

         if(cost[i][j] == 0)
            cost[i][j] = INF;
      }
   }

   visited[0] = 1;

   printf("Edges in MST:\n");

   while(edges < n - 1)
   {
      min = INF;
      for(i = 0; i < n; i++)
      {
         if(visited[i])
         {
            for(j = 0; j < n; j++)
            {
               if(!visited[j] && cost[i][j] < min)
               {
                  min = cost[i][j];
                  a = i;
                  b = j;
               }
            }
```

```
            }
        }

        printf("%d - %d = %d\n", a, b, min);

        minCost += min;

        visited[b] = 1;

        edges++;
    }

    printf("Minimum Cost = %d\n", minCost);

    return 0;
}
```

OUTPUT:



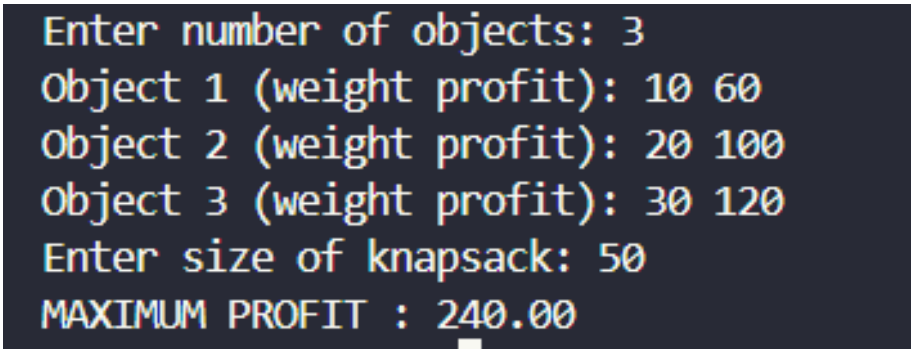
Lab program 9:

CODE:

```
#include <stdio.h>

struct object {
   int weight;
   int profit;
   float pwRatio;
};

void sort(struct object o[], int n) {
   struct object temp;
   for (int i = 0; i < n - 1; i++) {
      for (int j = 0; j < n - i - 1; j++) {
         if (o[j].pwRatio < o[j + 1].pwRatio) {
            temp = o[j];
            o[j] = o[j + 1];
            o[j + 1] = temp;
         }
      }
   }
}

int main() {
   int n;
   printf("Enter number of objects: ");
   scanf("%d", &n);

   struct object o[n];

   for (int i = 0; i < n; i++) {
      printf("Object %d (weight profit): ", i + 1);
      scanf("%d %d", &o[i].weight, &o[i].profit);
      o[i].pwRatio = (float)o[i].profit / o[i].weight;
   }

   sort(o, n);

   int size;
   printf("Enter size of knapsack: ");
   scanf("%d", &size);

   float totalProfit = 0.0;
   int rem_wt = size;

   int i = 0;
   while (rem_wt !=0) {
      if (o[i].weight <= rem_wt) {
         totalProfit += o[i].profit;
         rem_wt -= o[i].weight;
      } else {
         totalProfit += (o[i].profit * (float)rem_wt) / o[i].weight;
         rem_wt = 0;
```

```
        }
        i++;
    }

    printf("MAXIMUM PROFIT : %.2f\n", totalProfit);

    return 0;
}
```

OUTPUT:

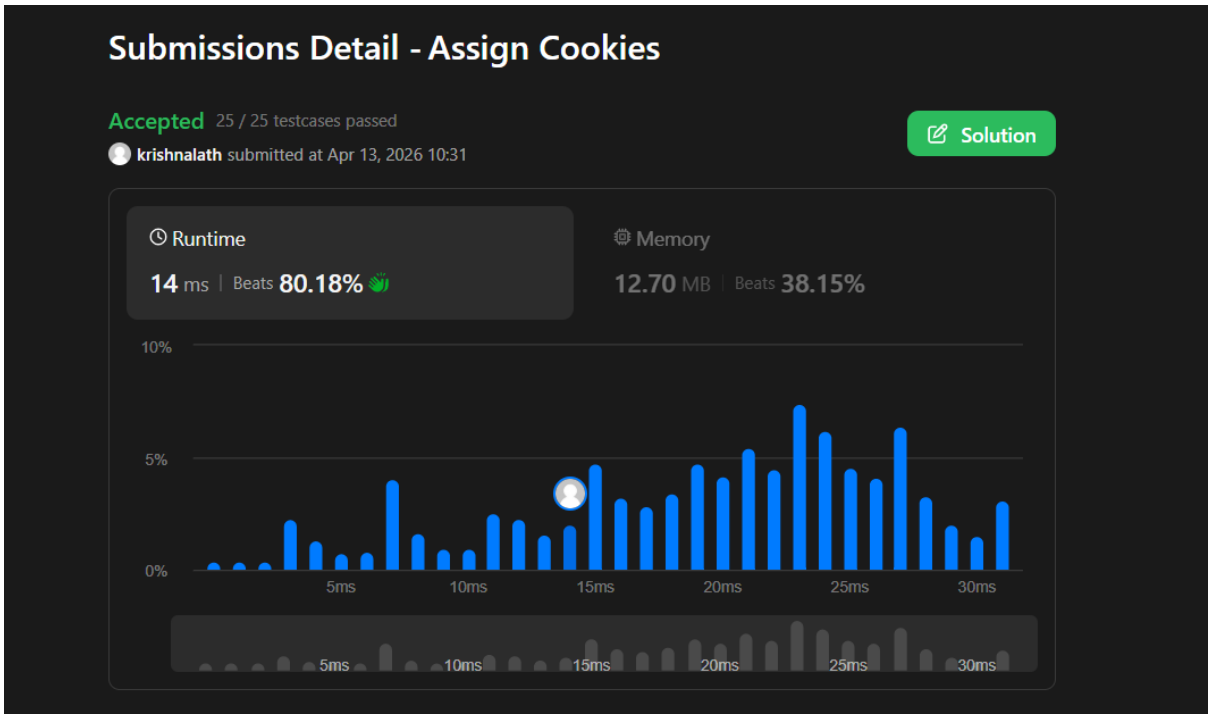


LEETCODE 455.Assign Cookies

CODE:

```
#include <stdio.h>
#include <stdlib.h>

int compare(const void *a, const void *b) {
    return (*(int*)a - *(int*)b); //dereferencing
}

int findContentChildren(int* g, int gSize, int* s, int sSize) {
    qsort(g, gSize, sizeof(int), compare); //greed
    qsort(s, sSize, sizeof(int), compare); //cookies
    int i = 0; // children
    int j = 0; // cookies
    int count = 0;

    while (i < gSize && j < sSize) {
        if (s[j] >= g[i]) {
            count++;
            i++;
            j++;
        } else {
            j++;
        }
    }
    return count;
}
```

OUTPUT:


Submissions Detail - Assign Cookies
Accepted 25 / 25 testcases passed
krishnalath submitted at Apr 13, 2026 10:31
Solution
Runtime
14 ms | Beats 80.18%
Memory
12.70 MB | Beats 38.15%
10%
5%
0%
5ms
10ms
15ms
20ms
25ms
30ms
5ms
10ms
15ms
20ms
25ms
30ms

Lab program 10:

CODE:

```
#include<stdio.h>
#include<limits.h>
void p(int parent[],int j){
   if(parent[j]==j){
      printf(" Path:%d ",j);
      return;
   }
   p(parent,parent[j]);
   printf("%d ",j);
}
void main(){
   int n;
   printf("Enter the number of vertices:");
   scanf("%d",&n);
   int a[n][n];
   printf("Enter the weights and enter 0 for no edges:\n");
   for(int i=0;i<n;i++){
      for(int j=0;j<n;j++){
         scanf("%d",&a[i][j]);
      }
   }
   int distance[n];
   int visited[n];
   int parent[n];
   int source;
   printf("Enter the source vertex:");
   scanf("%d",&source);
   for(int i=0;i<n;i++){
      distance[i]=(i!=source && a[source][i]==0)?INT_MAX:a[source][i];
      parent[i]=source;
      visited[i]=0;
   }
   distance[source]=0;
   visited[source]=1;
   for(int i=1;i<n;i++){
         int min=INT_MAX;
      int u=-1;
      for(int j=0;j<n;j++){
         if(!visited[j] && distance[j]<min){
            min=distance[j];
            u=j;
         }
      }
      if(u==-1) break;
      visited[u]=1;
      for(int v=0;v<n;v++){
         if(a[u][v]!=0 && !visited[v]){
            if(distance[u]+a[u][v]<distance[v]){
               distance[v]=distance[u]+a[u][v];
               parent[v]=u;
            }
```

```
            }

        }
    }
    printf("The distances are:\n");
    for(int i=0;i<n;i++){
        printf("%d to %d",source,i);
        if(distance[i]==INT_MAX) printf(":No path");
        else{
            p(parent,i);
            printf(":%d",distance[i]);
        }
        printf("\n");
    }
}
```

OUTPUT:

```
Enter number of vertices : 4
Enter adjacency matrix:
0 4 1 0
0 0 2 5
0 1 0 8
0 0 0 0
Enter source vertex : 0
Shortest paths from 0:
0 : 0
0 -> 2 -> 1 : 2
0 -> 2 : 1
0 -> 2 -> 1 -> 3 : 7
```

Lab program 11:

CODE:

```
#include <stdio.h>
#include <stdlib.h>

int x[20];
int place(int k,int i){
   for(int j=1;j<k;j++){
      if(x[j]==i || (abs(k-j)==abs(i-x[j])))
         return 0;
   }
   return 1;
}

void nQueens(int k,int n){
   for(int i=1;i<=n;i++){
      if(place(k,i)){
         x[k]=i;
         if(k==n){
               printf("\n");
            for(int j=1;j<=n;j++){
               printf("\nRow: %d , Column: %d",j,x[j]);
            }

         }else nQueens(k+1,n);
      }
   }
}

int main(){
   int n;
   printf("Enter number of queens: ");
   scanf("%d",&n);
   nQueens(1,n);
   return 0;
}
```

OUTPUT:

```
Enter number of queens: 4


Row: 1 , Column: 2
Row: 2 , Column: 4
Row: 3 , Column: 1
Row: 4 , Column: 3

Row: 1 , Column: 3
Row: 2 , Column: 1
Row: 3 , Column: 4
Row: 4 , Column: 2
```